



**UNIVERSIDAD
DE GRANADA**

**Escuela Técnica Superior de Ingenierías Informática
y Telecomunicaciones (ETSIIT).**

Grado en Ingeniería Informática.

Alumno: Carlos Alberto Rodríguez Ferreira.

**Asignatura: Periféricos y Dispositivos de Interfaz
Humana.**

Curso Académico: 2025/26.

**Práctica 1: Entrada/Salida utilizando
interrupciones con lenguaje C/C++.**

Gotoxy():

Para la elaboración de este ejercicio he usado las librerías dos.h y stdio.h.

La librería dos.h la he usado para poder hacer uso de las interrupciones, entradas y demás, y la librería stdio.h para poder mostrar en pantalla y capturar desde teclado.



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program:
- File Edit Search Run Compile Debug Project Options
[*] GOTOXY.C
#include <dos.h>
#include <stdio.h>

void gotoxy(int fila, int columna){
    union REGS inregs, outregs;
    inregs.h.ah = 2;
    inregs.h.bh = 0;
    inregs.h.dh = fila;
    inregs.h.dl = columna;
    int86(0x10, &inregs, &outregs);
}

int main (){
    int valorx;
    int valory;

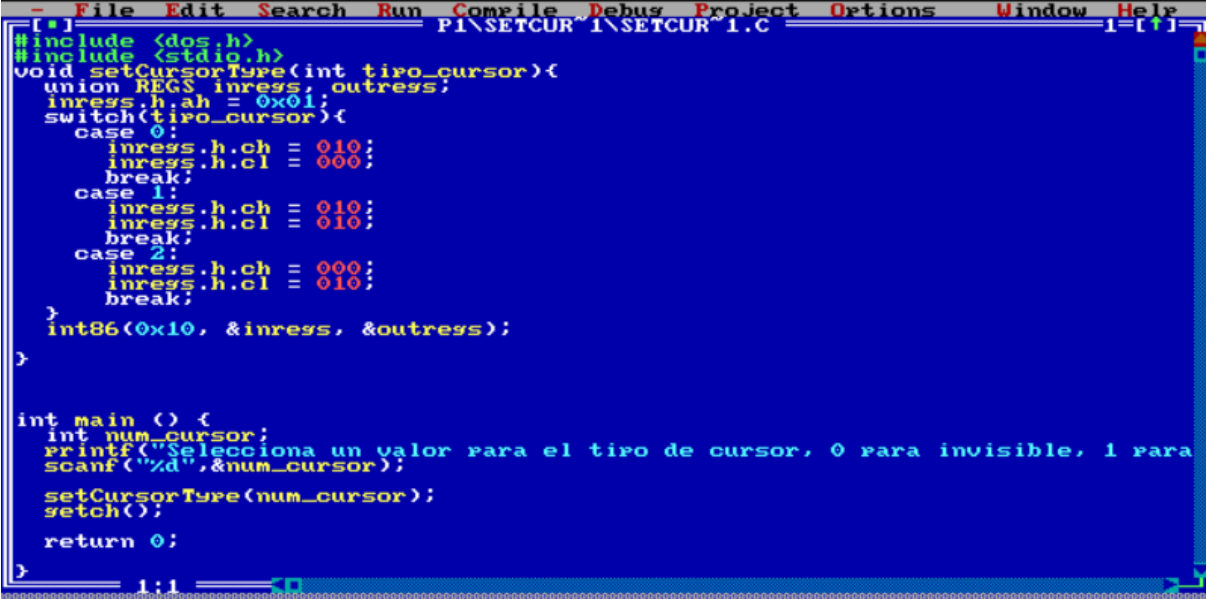
    printf("Introduce un valor para fila");
    scanf("%d", &valorx);
    while(valorx >= 25){
        printf("Introduce un valor valido - inferior a 25 -:_");
        scanf("%d", &valorx);
    }

    printf("Introduce un valor para columna");
    scanf("%d",&valory);
    while(valory >= 80){
        printf("Introduce un valor valido - inferior a 80 -: ");
        scanf("%d", &valory);
    }

    gotoxy(valorx, valory);
    getch();
    return 0;
}
```

La función getch y scanf son propias de stdio.h. getch() es para poder congelar la pantalla de forma que se pueda ver donde se ha movido el cursor, y la función scanf() es para poder recoger de teclado los valores introducidos. Es básicamente el cin de c++.

SetCursorType():



```
[*] File Edit Search Run Compile Debug Project Options Window Help
P:\SETCUR 1\SETCUR 1.C 1=111
#include <dos.h>
#include <stdio.h>
void SetCursorType(int tipo_cursor){
    union REGS inregs, outregs;
    inregs.h.ah = 0x01;
    switch(tipo_cursor){
        case 0:
            inregs.h.ch = 010;
            inregs.h.cl = 000;
            break;
        case 1:
            inregs.h.ch = 010;
            inregs.h.cl = 010;
            break;
        case 2:
            inregs.h.ch = 000;
            inregs.h.cl = 010;
            break;
    }
    int86(0x10, &inregs, &outregs);
}

int main () {
    int num_cursor;
    printf("Selecciona un valor para el tipo de cursor, 0 para invisible, 1 para
    scanf("%d",&num_cursor);

    setCursorType(num_cursor);
    getch();

    return 0;
}
```

Para la realización de este apartado he vuelto a incluir la librería `stdio.h`, además de la librería `dos.h` que es esencial para su realización, para poder recoger datos del teclado a través de `scanf()`, poder usar `printf()` y poder congelar la pantalla con `getch()` para poder apreciar el resultado.

SetVideoMode():

```
- File Edit Search Run Compile Debug Project Options
[.] SETVIDEO.C
#include <dos.h>
#include <stdio.h>
#include <conio.h>
void setVideoMode(int tiroVideo){
    union REGS inress, outress;
    inress.h.ah = 0x00;
    switch(tiroVideo){
        case 0:
            inress.h.al = 0x03;
            break;
        case 1:
            inress.h.al = 0x00;
            break;
        case 2:
            inress.h.al = 0x13;
            break;
        case 3:
            inress.h.al = 0x0D;
            break;
    }
    int86(0x10, &inress, &outress);
}

int main (){
    int tirovideo;
    printf("Introduzca el numero del modo de video que desea");
    printf("\n0 Texto normal");
    printf("\n1 Texto reducido");
    printf("\n2 Grafico con 256 colores");
    printf("\n3 Grafico con 16 colores\n");
    scanf("%d",&tirovideo);
    setVideoMode(tirovideo);
    getch();
    return 0;
}
```

He usado un switch para que se guarden los registros según qué opción, además de usar las librerías dos.h, stdio.h para lo comentado anteriormente.

GetVideoMode();

```
- File Edit Search Run Compile Debug Project Options Window Help
[.] GETVIDEO.C
#include <dos.h>
#include <stdio.h>
#include <conio.h>
char* setVideoMode(){
    int modo;
    union REGS inress, outress;
    inress.h.ah = 0x0F;
    int86(0x10, &inress, &outress);
    modo = outress.h.al;
    if(modo == 0x00 || modo == 0x01 || modo == 0x02 || modo == 0x03 || modo ==
    } else {
        return "modo grafico";
    }
}

int main(){
    printf("El modo de vido actual es %s", setVideoMode());
    getch();
    return 0;
}
```

Para este ejercicio lo que he hecho ha sido obtener el modo de vídeo una vez lanzada la interrupción int86 y he hecho un if en el que si devolvía el código de cada uno de los modos texto que hay, devolviera un texto de modo texto y todo lo demás devolviera modo gráfico.

TextColor():

```

1  [●] TEXTCOLO.C 1=[1]
2  #include <dos.h>
3  #include <stdio.h>
4  #include <conio.h>
5
6  unsigned char color_actual = 15;
7
8  void textColor(unsigned char color){
9      union REGS inress, outress;
10     inress.h.ah == 0x09;
11     inress.h.al == 1;
12     inress.h.bh == 0x00;
13     inress.h.bl == color;
14     inress.x.cx == 1;
15
16     int86(0x10, &inress, &outress);
17 }
18
19 int main (){
20     int color;
21     int bucle = 1;
22
23     char texto[100];
24
25     printf("Elige un color");
26     printf("\n 0.Negro \n 1.Azul \n 2.Verde \n 3.Cian \n 4.Rojo \n 5.Masenta \n 6");
27     scanf("%i", &color);
28
29     textColor(color);
30     while (bucle == 1){
31         cprintf("\r\n El texto se encuentra del color que se ha seleccionado");
32         scanf("%s", texto);
33     }
34 }

```

En este ejercicio, para poder cambiar el color del texto, lo que he hecho ha sido crear una función a la que se pasa una variable que contiene el código del color que se quiere ver en el terminal.

TextBackground():

```

1=[1]=
BACKGROU.C
#include <dos.h>
#include <stdio.h>
#include <conio.h>

void textBackgroundColor(char color){
    union REGS inress, outress;

    inress.h.ah= 0x09;
    inress.h.al = 7;
    inress.h.bh =0x00;

    inress.h.bl = color << 4;
    inress.x.cx = 1;

    int86(0x10, &inress, &outress);
}

int main(){
    int color;
    int bucle = 1;
    char texto[100];

    printf("Elige un color de fondo");
    printf("\n 0.Negro \n 1.Azul \n 2.Verde \n 3.Cian \n 4.Rojo \n .Magenta \n");
    scanf("%i",&color);

    textBackgroundColor(color);

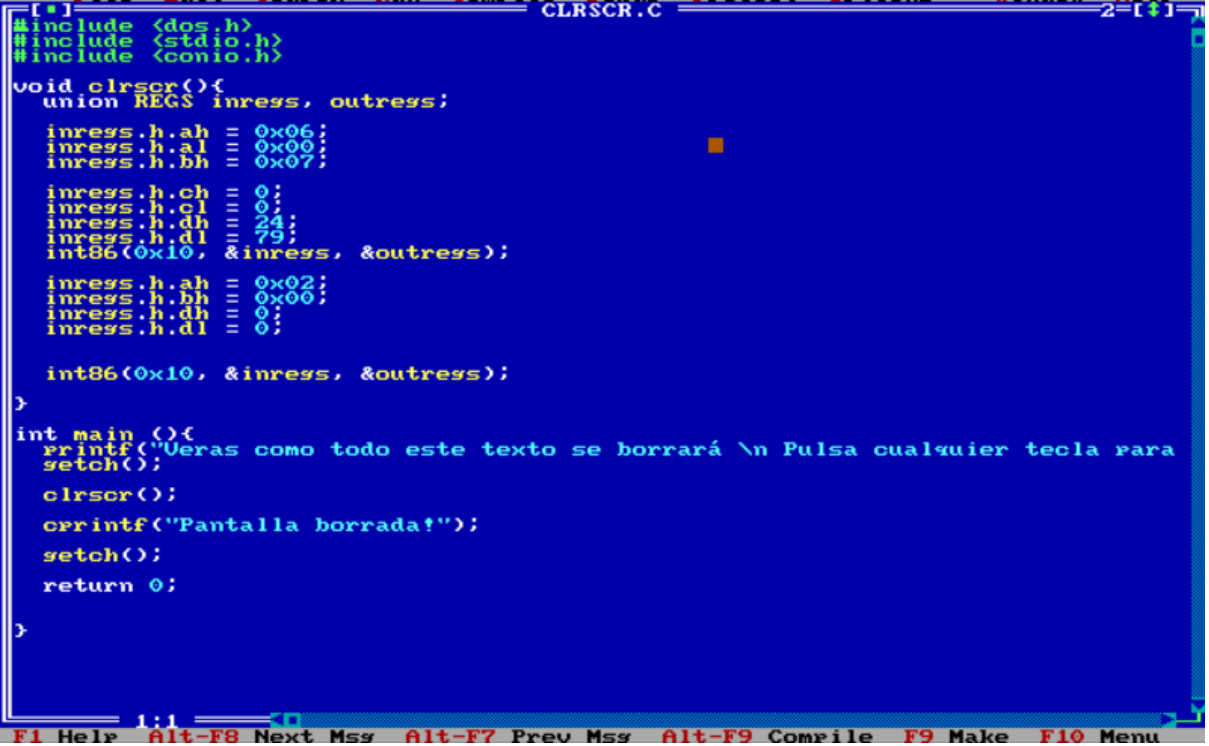
    while(bucle ==1){
        printf("\n Se ha pintado el fondo del color seleccionado, puede escribir ");
        scanf("%s",&texto);

        printf("\n desea seguir escribiendo?\n 1.Si \n 2.No");
        scanf("%i",&bucle);
    }
}

```

Para este ejercicio he usado la interrupción 10h, dentro de los registros he modificado el color en el primer cuarteto de byte, correspondientes a los 4 primeros dígitos de izquierda a derecha.

clrscr():



```
1:1 CLRSCR.C
#include <dos.h>
#include <stdio.h>
#include <conio.h>

void clrscr(){
    union REGS inregs, outregs;

    inregs.h.ah = 0x06;
    inregs.h.al = 0x00;
    inregs.h.bh = 0x07;

    inregs.h.ch = 0;
    inregs.h.cl = 0;
    inregs.h.dh = 24;
    inregs.h.dl = 79;
    int86(0x10, &inregs, &outregs);

    inregs.h.ah = 0x02;
    inregs.h.bh = 0x00;
    inregs.h.dh = 0;
    inregs.h.dl = 0;

    int86(0x10, &inregs, &outregs);
}

int main(){
    printf("Veras como todo este texto se borrará \n Pulsa cualquier tecla para\n");
    clrscr();
    cprintf("Pantalla borrada!");
    getch();
    return 0;
}
```

En este he tenido que realizar dos interrupciones int86. Una para poder limpiar la pantalla totalmente y la otra para poder desplazar el cursor hacia el inicio del todo.(En este ejercicio me he apoyado de la ayuda de Gemini).

cputchar():

No entendí realmente qué era lo que pedía el ejercicio. Por eso no lo he realizado

getche():

```
- File Edit Search Run Compile Debug Project Options Window Help
[.] GETCHE.C 3=1+1
#include <dos.h>
#include <stdio.h>
#include <conio.h>

char mi_getche(){
    union REGS inregs, outregs;
    char tecla;

    inregs.h.ah=0x00;
    int86(0x16, &inregs, &outregs);
    tecla = outregs.h.al;

    inregs.h.ah = 0x0E;
    inregs.h.al = tecla;
    inregs.h.bh = 0x00;
    int86(0x10, &inregs, &outregs);
    return tecla;
}

int main(){
    char resultado;
    printf("Pulsa cualquier tecla");
    resultado = mi_getche();
    printf("\nSe ha recibido la tecla \n", resultado);
    getch();
    return 0;
}
```

Para realizar este ejercicio, he hecho dos interrupciones, una para recoger el carácter una vez presionado sin necesidad de pulsar enter y la segunda interrupción ha sido para poder sacarlo en pantalla.

pixel():

```
- File Edit Search Run Compile Debug Project Options
[.] PIXEL.C
#include <dos.h>
#include <stdio.h>
#include <conio.h>

void pixel(int x, int y, char color){
    union REGS inregs, outregs;

    inregs.h.ah = 0x0C;
    inregs.h.al = color;
    inregs.h.bh = 0x00;
    inregs.x.cx = x;
    inregs.x.dx = y;
    int86(0x10, &inregs, &outregs);
}

int main(){
    union REGS inregs, outregs;

    inregs.h.ah = 0x00;
    inregs.h.al = 0x13;
    int86(0x10, &inregs, &outregs);

    pixel(160,100,4);
    pixel(161,100,3);
    pixel(160,101,5);
    pixel(161,101,6);

    getch();

    inregs.h.ah=0x00;
    inregs.h.al=0x03;
    return 0;
}
```

En este ejercicio, lo que hice fue crear la función pixel que solamente se encarga de poner el pixel según la posición que le pasamos y el color que queremos que tenga. En el main

hemos hecho toda la gestión de gráfica y una vez que hemos dejado de hacer cosas en el modo gráfico, volvemos al modo texto.